

Strain Scores: Your Heart Rate Doesn't Know You Did Squats

A Strain score should answer the simple question: “how much load did my body take on today?”. This is different from readiness, as readiness asks whether the body looks prepared for strain; strain asks how much stress was actually accumulated. This means that the score should be driven mainly by exercise intensity, exercise duration, heart rate response, and activity load, with extra consideration for strength training and other work that does not show up perfectly through heart rate. The mainstream wearable companies mostly agree with this, but they all expose the logic differently.

WHOOP has made Strain one of its main scores. The publicly given explanation is centred mainly around cardiovascular strain, especially how long the user spends at elevated heart rates. This makes sense for cardio-heavy activity, however WHOOP has more recently added muscular load because pure heart-rate strain tends to undercount strength training. This is an important point for our app. A heavy leg session can create real fatigue even if heart rate is not elevated for long enough to look like a hard run. [\[1\]](#)

Apple's Training Load is also useful because it separates today's workout effort from the longer-term interpretation of whether load is ramping up. Apple uses a 1-10 effort rating after workouts and automatically estimates effort for many cardio workouts using data like heart rate, GPS, elevation, age, height and weight, and compares the most recent 7 days with the previous 28 days. This is probably the cleanest mainstream version of the idea: first estimate how hard the activity was, then compare recent load to what the person's baseline is. [\[2\]](#)

Garmin has a wider training ecosystem, including acute load, load focus, recovery time and Training Readiness. Garmin's most useful contribution is that load is not just “calories burned” or “minutes exercised”. It is interpreted in the context of intensity, recovery and whether the user is building or overdoing training. [\[3\]](#)

Google's Cardio Load paper defines Cardio Load as cardiovascular work accumulated across the day, based on heart-rate reserve (calculated as maximum heart rate - resting heart rate) and capturing both intensity and duration. This is a strong basis for a strain score because heart-rate reserve is more personalised than just using raw heart rate. A heart rate of 150 bpm does not mean the same thing for everyone because it may represent a relatively low percentage of one person's cardiovascular capacity but a much higher percentage of another's, depending on factors such as their resting heart rate, maximum heart rate, age, and fitness level. [\[4\]](#)

Moving to the scientific literature, the main history here starts with training impulse models like TRIMP. These models attempt to summarise training load from intensity and duration of exercise, usually using heart rate. The basic idea is still correct: the longer you spend at a higher intensity, the more strain you gain. Foster's session-RPE work adds another useful layer by showing that perceived effort can be a practical way to quantify training load, especially when heart rate alone misses some of the picture. [\[5\]](#)
[\[6\]](#)

The main scientific warning is that while strain is good at measuring exposure it is worse at perfectly predicting injury or recovery. Acute workload ratio models are useful because they compare recent load with what the athlete is used to, but review papers have shown problems with treating the ratio as a magic injury predictor. The ratio can be statistically unstable, sensitive to how it is calculated, and confounded by lots of things that we don't measure. This means we should use recent-versus-chronic load as an interpretation tool, not pretend that it gives us an exact risk number. [\[7\]](#)

The most important feature for strain is cardiovascular intensity multiplied by duration. This should be the largest segment of the score. Google's Cardio Load does this through heart-rate reserve, and that is likely the right direction. Time in higher heart-rate zones should count much more than time in very low zones, but lower intensity activity should still count if there is enough of it. A long walk shouldn't look like a hard interval session, but it shouldn't be ignored either. [\[4\]](#)

The 2nd important feature is external work: distance, pace, elevation, speed, steps, power where available, and workout type. This matters as heart rate is influenced by heat, stress, caffeine, illness and fatigue. It also responds slowly to some forms of exercise. A hill run, a flat run, and a cycling session can produce different mechanical and cardiovascular demands even if the average heart rate looks similar. Apple using GPS and elevation in automatic effort estimation supports this idea. [\[2\]](#)

The 3rd important feature is muscular load. This is the part that pure cardio models miss. Strength training, hill walking, loaded carries, sprinting, and stop-start sport can create large local muscular fatigue without producing the same heart-rate pattern as steady-state cardio. If we only use heart rate, the app will probably underrate gym sessions and some sports. For a basic first version, muscular load could be estimated from workout type, duration, movement patterns, accelerometer data if available, and maybe user-entered effort. A more advanced version would need to understand what exercises were actually performed. This would likely require users to enter workout data such as exercises, sets, reps, weight, and perceived effort. This would let the app distinguish between a light upper-body session and a heavy lower-body session, even if

the heart-rate data looked similar. For example, 5 sets of heavy squats should create a very different muscular load than 3 sets of light curls. This would make the strain score much better at capturing the kind of fatigue that heart-rate-only models miss.

The 4th important feature is subjective effort. This feels less technical, but it is actually very useful. A user knows when a workout feels unusually hard. Apple lets users manually adjust effort for things like stress or soreness, and Foster's session-RPE work supports the idea that perceived effort can be a valid training-load input. For this app, even a simple 1-10 "how hard did this feel?" question after workouts would improve the score, especially for strength training. [\[2\]](#) [\[6\]](#)

In terms of rough importance, we could structure the Strain calculation like this, but as load points rather than a 0-100 score:

- Cardiovascular intensity x duration: around 50%. This is the backbone of the score.
- External work: around 15%. Distance, pace, elevation, speed, steps or power where available.
- Muscular load: around 15%. Especially important for strength training and non-steady-state activity.
- Subjective effort / RPE: around 10%. Useful as it captures things sensors miss.
- Sport-specific correction: around 10%. Workout type should change how we interpret the same raw signals.

This weighting should be personalised. A 45-minute run at 155 bpm is not the same strain for a beginner as it is for a trained runner. This is why heart-rate reserve is better than raw heart rate, and why the app should learn personal baselines over time. Strain should compare activity to the person's own fitness, resting heart rate, max heart rate estimate, normal weekly load and usual response to workouts.

The app should also separate raw strain from strain interpretation. This is important. The raw Strain score should say "how much load did I accumulate today?". It should not be lowered just because the user slept badly. Poor sleep belongs to readiness. However, the interpretation of strain should consider readiness and be put into context. For example, the app might say:

"Today's strain was moderate, but since readiness was low, this may have a higher recovery cost than usual."

That keeps the numbers clean. Strain remains a load score. Readiness remains a recovery score. The insight layer explains how they interact.

Strain should not be calculated as a normal 0-100 score. That framing makes sense for quality-based scores, where 100 can roughly mean “excellent” and 0 means “very bad”. Strain is different as it is a load dose, meaning it measures how much stress the body accumulated. There is no clear maximum where 100 would mean “complete”, and setting one would be arbitrary. If 100 meant “weekly target reached”, then the app would struggle to show whether the user slightly exceeded their target or massively overdid it. If 100 meant “maximum possible strain”, then almost nobody would know what that actually means (including me). So a bounded 0-100 score is probably the wrong structure. The better strategy is to calculate Strain as accumulated load points. This is closer to Google’s Cardio Load approach, where the user accumulates load across the day and week based on cardiovascular work.

This means our app should have two related numbers:

- Daily Strain: how much load the user accumulated today.
- Weekly Strain Target: how much load the user should aim for this week based on their recent normal training load.

This is much clearer than forcing Strain into 0-100. For example, if the user’s weekly target is 150 and they have accumulated 90 so far, the app can show that they are 60% of the way there. If they reach 180, the app can show that they are 120% of target and explain whether this seems like a productive increase or a possible overreach. This is more useful than just showing “100/100”, because going above target is an important thing that we actually need to understand.

This weekly target should be personalised. Google’s paper argues for weekly targets because daily training naturally fluctuates: some days are hard, some days are easy, and some days are rest days. A weekly target smooths this out and fits better with how people actually train. The target should be based on the user’s chronic load, for example their recent 28-day average or a mixture of rolling average and exponentially weighted moving average. The goal isn’t to force the same number every week, but to estimate the amount of load that maintains or gradually improves the user’s current fitness without suddenly spiking above what they are used to. [\[4\]](#)

Apple’s version also supports the same idea. They compare the last 7 days of training load with the last 28 days, then classifies the user as below, steady, above, or well above their regular load. This is the right interpretation layer for our app as well. The app should not just say “you did 40 strain today”. It should say if that is low, normal, high, or unusually high for this person. [\[2\]](#)

This gives us the cleanest version of the metric. Strain becomes a measure of how much work the body actually had to handle, while the app's guidance explains whether that amount of work is productive, normal, too low, or potentially too much.